

Final draft

1. Design Requirements

1.1 Problem Statement

- Mapping of Conv1 layer for simplified LeNet-5
- Input: 8×8 grayscale image (single channel)
- Kernel: 3×3 convolution
- Stride: 2
- Padding: 1
- Output: 4×4 feature map (16 values)

1.2 Hardware Constraints

- 4 MAC units (Multiply-Accumulate)
- Each MAC: 8-bit × 8-bit signed integer multiplication
- 16 bytes on-chip SRAM (dual-port, each port 32-bit = 4 bytes/cycle)
- External memory bandwidth: 1 KB/s (1024 bytes/second)
- No additional hardware (reuse MAC adders for reduction)

1.3 Design Goals

- Maximize throughput (images/second)
- Minimize on-chip memory footprint
- Efficient bandwidth utilization
- Minimize MAC idle time through pipelining

1.4 Data Types

- Input pixels: 8-bit signed integers [-128, 127]
- Weights: 8-bit signed integers [-128, 127]
- Products: 16-bit signed integers
- Accumulator: 20-bit signed integers (to prevent overflow)
- Output: 20-bit signed integers (3 bytes packed)

2. Architecture Assumptions

2.1 Memory assumptions

2.1.1 Memory Assumptions

- External memory is byte-addressable
- Row-major storage for input image
- 8-byte parallel read capability from SRAM
- Weight read for every patch

2.1.2 Dual-Port Access

- Port A: 32-bit read/write (4 bytes simultaneously)
- Port B: 32-bit read/write (4 bytes simultaneously)
- Both ports can operate in same cycle
- A dual-port SRAM provides **two independent address ports**, NOT two bytes from one port:

```SRAM Layout (16 bytes total):

|         |             |         |             |
|---------|-------------|---------|-------------|
| +-----+ |             | +-----+ |             |
|         | Bank A (8B) |         | Bank B (8B) |
| +-----+ |             | +-----+ |             |
|         | Bytes 0-7   |         | Bytes 8-15  |
| +-----+ |             | +-----+ |             |

### 2.1.3 Data Loading Strategy

Since SRAM (16 bytes) cannot hold entire image (64 bytes), data is **streamed**:

1. **Image pixels**: Loaded on-demand per patch, not stored entirely
2. **Weights**: fetched per batch  $4 + 4 + 1$  bytes = 9 bytes
3. **Working set per patch**: 9 weights + 9 pixels = 18 bytes
  - Exceeds 16-byte SRAM
  - 3 batches, 8 bytes per batch, no full 18B stored at once

## 2.2 Timing Assumptions

- Multiplier latency: 2 cycles (pipelined)
- Adder latency: 3 cycles (matching FP32 timing)
- SRAM access: up to 8 bytes/cycle aggregate (4 bytes per port  $\times$  2 ports).
- Register load: Included in operation timing

## 2.3 Arithmetic Assumptions

- Signed multiplication using Booth encoding or similar
- Two's complement representation
- 20-bit accumulator sufficient for worst-case:
  - Max value of data per product:  $127 \times 127 = 16,129$
  - Max value of 3 products in ACC0:  $3 \times 16,129 = 48,387$

- Fits in 20 bits (max value of 20 bits signed = 524,287)
- No saturation needed for integer operations

## 2.4 Control Assumptions

- Synchronous design, single clock domain
- Sequential processing of patches (no inter-patch pipelining)
- Intra-batch operations can overlap (load, multiply, add)
- Reduction uses MAC adders in ADD-only mode

## 2.5 Hardware Assumptions

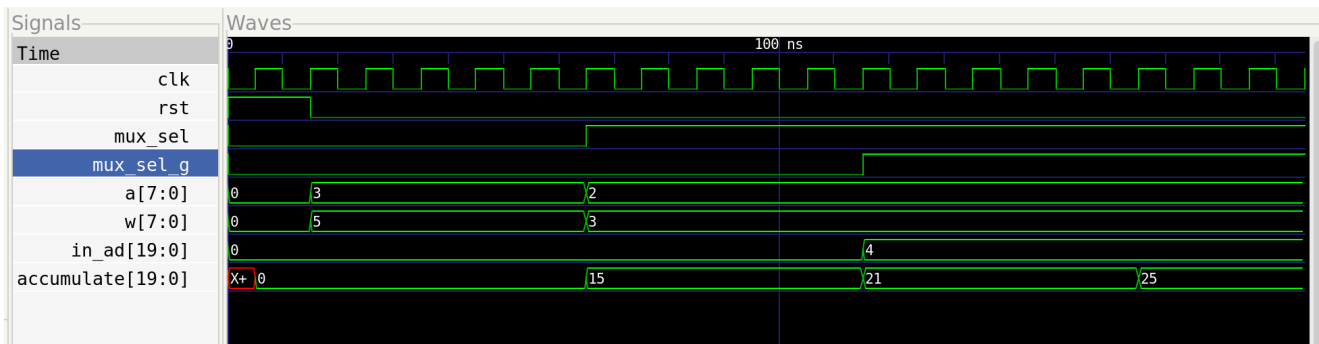
- MAC units can operate in two modes:
  - MAC mode: multiply-accumulate
  - ADD-only mode: bypass multiplier, add 20-bit inputs
- MUX overhead is negligible in critical path
- Multiplier and adder are combinational with stated latencies

## 3. Theory of Operations

### COMPLETE PATCH COMPUTATION

| Cycle | Batch | Operation            | MAC0  | MAC1  | MAC2  | MAC3  | Notes                |
|-------|-------|----------------------|-------|-------|-------|-------|----------------------|
|       | -     | Loading SRAM         |       |       |       |       | Loading weights      |
|       | -     | Loading SRAM         |       |       |       |       | Loading patch values |
| 0     | 0     | Load from SRAM       | w0,a0 | w1,a1 | w2,a2 | w3,a3 | Parallel load        |
| 1     | 0     | Multiply (cycle 1/2) | x     | x     | x     | x     | Pipeline stage 1     |
| 2     | 0     | Multiply (cycle 2/2) | x     | x     | x     | x     | Products ready       |
| 3     | 0     | Add (cycle 1/3)      | +     | +     | +     | +     | Start accumulate     |
| 4     | 0     | Add (cycle 2/3)      | +     | +     | +     | +     | Continue             |
| 5     | 0     | Add (cycle 3/3)      | +     | +     | +     | +     | ACC updated (reset)  |
| 6     | 1     | Load from SRAM       | w4,a4 | w5,a5 | w6,a6 | w7,a7 | Next batch           |

| Cycle | Batch | Operation            | MAC0  | MAC1 | MAC2 | MAC3 | Notes                                      |
|-------|-------|----------------------|-------|------|------|------|--------------------------------------------|
| 7     | 1     | Multiply (cycle 1/2) | ×     | ×    | ×    | ×    |                                            |
| 8     | 1     | Multiply (cycle 2/2) | ×     | ×    | ×    | ×    |                                            |
| 9     | 1     | Add (cycle 1/3)      | +     | +    | +    | +    | Accumulate mode                            |
| 10    | 1     | Add (cycle 2/3)      | +     | +    | +    | +    |                                            |
| 11    | 1     | Add (cycle 3/3)      | +     | +    | +    | +    | ACC += product                             |
| 12    | 2     | Load from SRAM       | w8,a8 | idle | idle | idle | Last operand                               |
| 13    | 2     | Multiply (cycle 1/2) | ×     | idle | idle | idle | MAC0 only                                  |
| 14    | 2     | Multiply (cycle 2/2) | ×     | idle | LOAD | idle | Loading value from ACC3 to ACC2 to overlap |
| 15    | 2     | Add (cycle 1/3)      | +     | idle | +    | idle |                                            |
| 16    | 2     | Add (cycle 2/3)      | +     | idle | +    | idle |                                            |
| 17    | 2     | Add (cycle 3/3)      | +     | idle | +    | idle |                                            |
| 18    | Red   | Reduction step 2     | idle  | +    | idle | idle | ACC1 +ACC2                                 |
| 19    | Red   | (cycle 2/3)          | idle  | +    | idle | idle |                                            |
| 20    | Red   | (cycle 3/3)          | idle  | +    | idle | idle |                                            |
| 21    | Red   | Reduction step 3     | +     | idle | idle | idle | ACC0+ACC1                                  |
| 22    | Red   | (cycle 2/3)          | +     | idle | idle | idle |                                            |
| 23    | Red   | (cycle 3/3)          | +     | idle | idle | idle | Final sum ready                            |



**Result:** Final sum in ACC0

Can write to output FIFO / memory in cycle 24.

**Key:**  $\times$  = Multiply in progress ·  $+$  = Add/accumulate in progress · LOAD = Loading external ACC value for ADD-only mode · idle = MAC not active

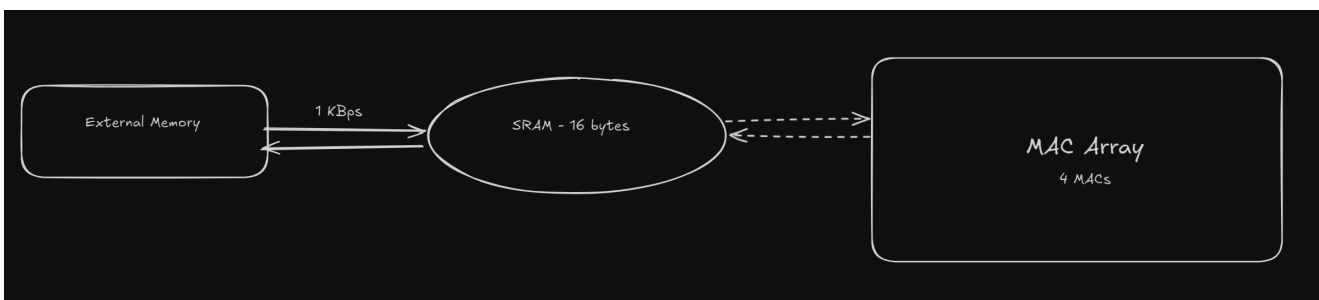
## Summary of operations

Per image computation:

- 16 patches (4×4 output grid)
- Each patch: 24 cycles
- Total:  $16 \times 24 = 384$  cycles per image
- Memory access per image:
  - Input pixels: 64 bytes (8×8, loaded once)
  - Outputs: 16 patches \* 3 bytes = 48 bytes
  - weights called for every patch:  $9 \times 16 = 144$
  - Total:  $64 + 144 + 48 = 256$  bytes per image
- Throughput:
  - Bandwidth: 1024 bytes/s
  - Images/s:  $1024 / 256 \approx 4$  images/s
- Clock frequency:
  - Cycles per image: 384
  - Required clock:  $384 \times 4 \approx 1536 = 1.5$  kHz

## 4. Architecture

### 4.1 System-Level Block Diagram

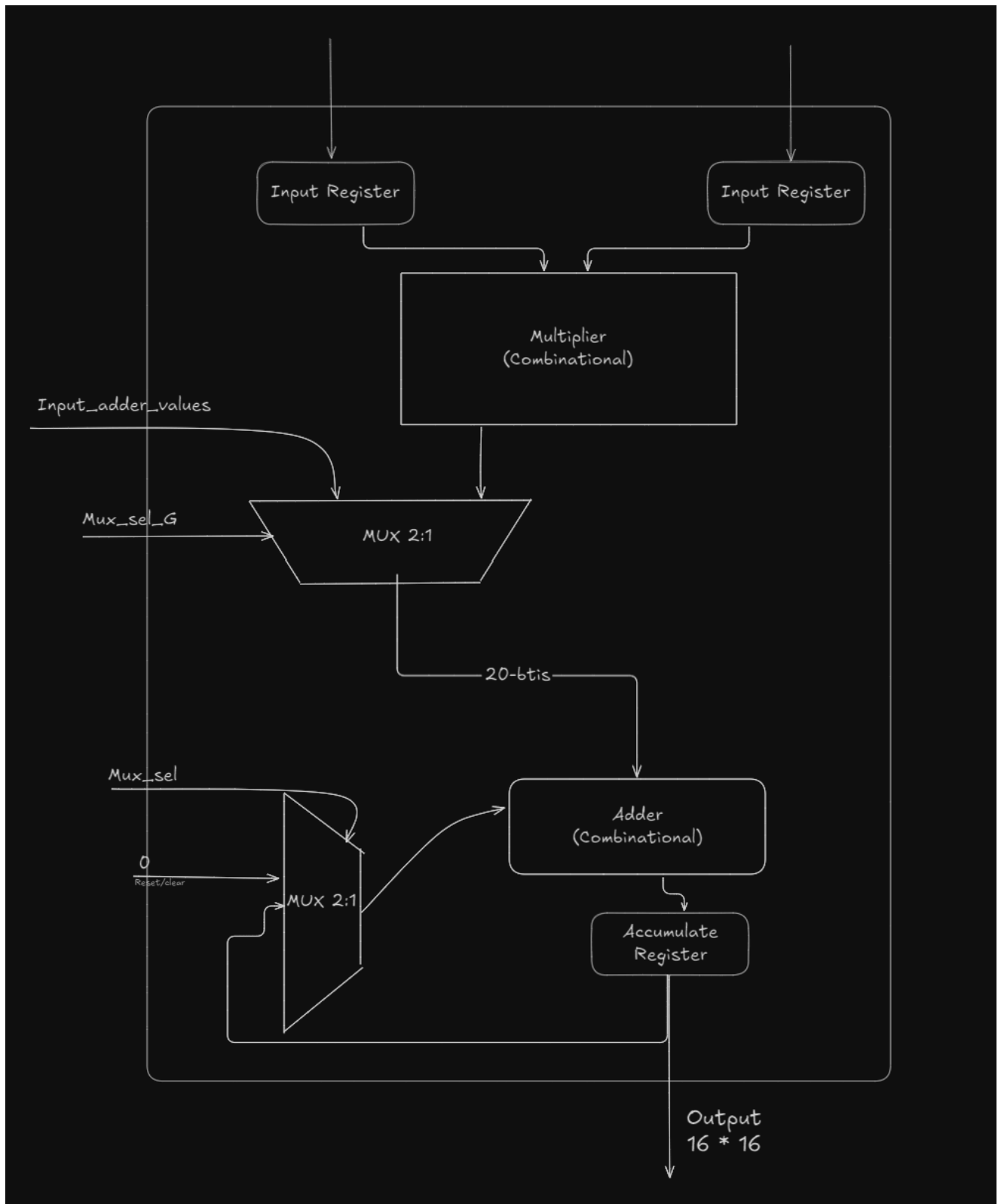


#### 4.1.1 Data Flow

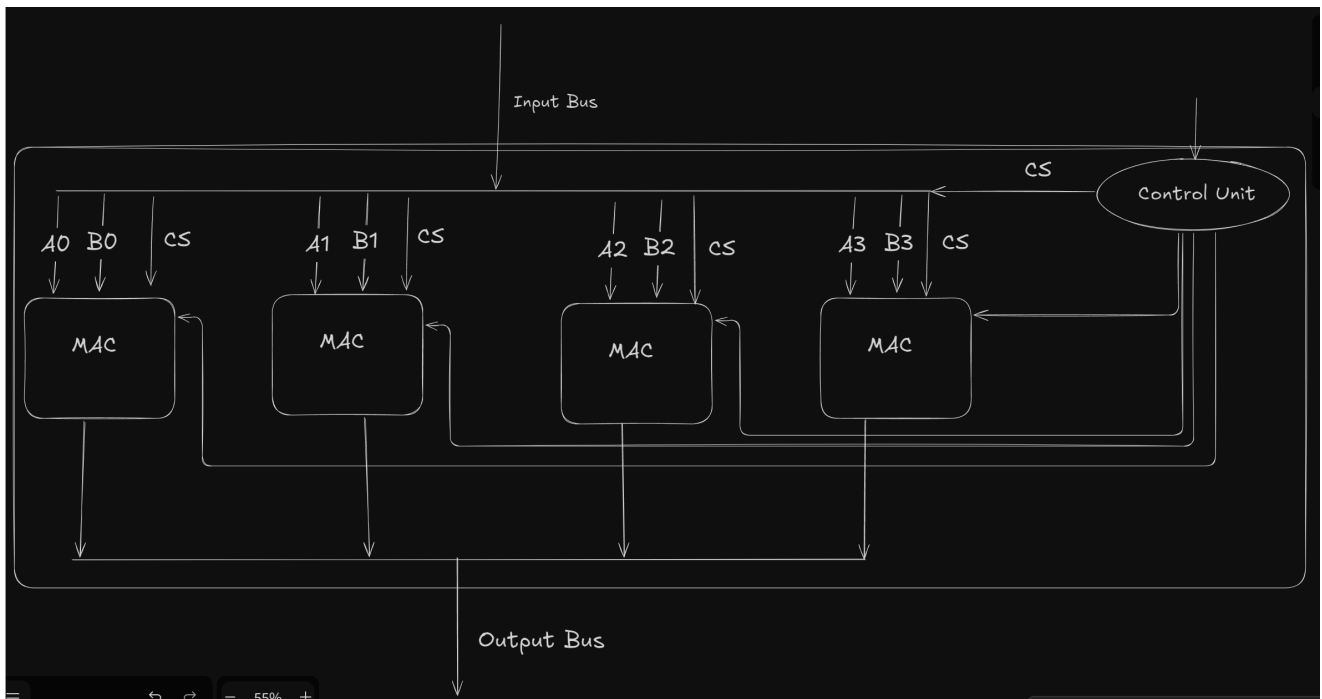
Explain step-by-step with arrows:

1. External memory → SRAM (weights + pixels)
2. SRAM → MAC input registers
3. Registers → Multiplier → Product
4. Product → Adder (with ACC feedback)
5. Adder → ACC register
6. For reduction: ACC → MUX → Adder (bypass mult)
7. Final ACC → Output

## 4.2 MAC Unit Architecture



### 4.3 MAC Array (4 units)

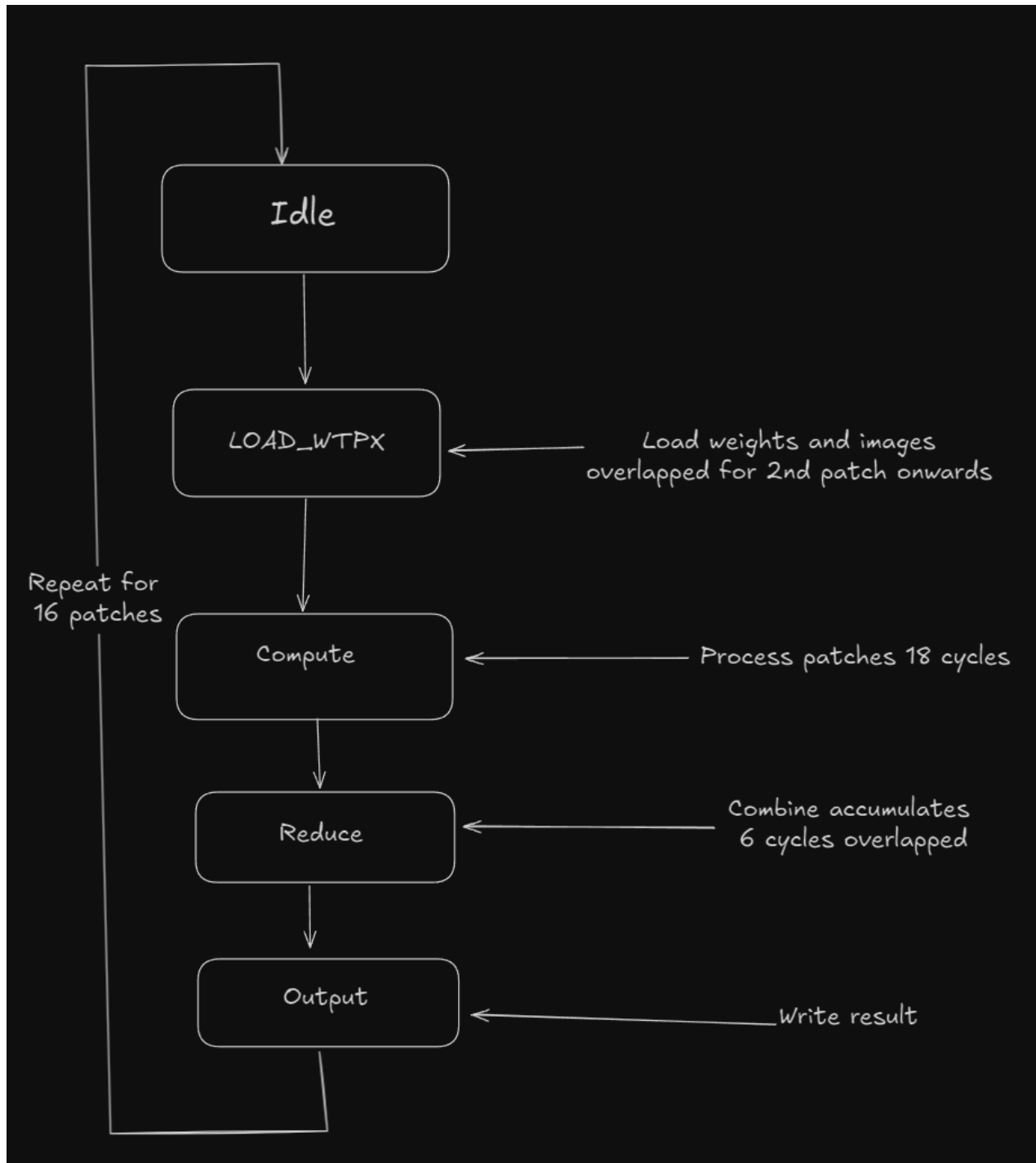


## 4.4 Control Unit

- Generates control signals based on current cycle
- Manages batch sequencing (0, 1, 2)
- Switches MAC modes (MAC vs ADD-only)
- Handles SRAM addressing



- Coordinates external memory access



## 4.5 Critical Signals

List main control signals:

- clk: System clock (~1.5 kHz)
- reset: Global reset
- mode[1:0]: MAC operation mode
- mux\_sel\_G[3:0]: MUX select for each MAC
- acc\_reset[3:0]: Clear accumulator
- sram\_addr[3:0]: SRAM address
- sram\_we: SRAM write enable

- ext\_mem\_rd: External memory read
- ext\_mem\_wr: External memory write

## 5. Final Notes and Summary

### 5.1 Pipeline Stalls & Hazards

Even though the schedule is fixed, a few practical issues can introduce stalls:

- Memory stalls: external bandwidth variation, SRAM port contention, and patch reloads can delay MAC start times.
- Compute stalls: multiplier/adder latency mismatch or accumulator dependencies can bubble the pipeline.
- Control hazards: off-by-one transitions in batch switching or incorrect address generation can corrupt patch outputs.

These don't break the architecture but require tight control alignment and predictable memory behaviour.

### 5.2 Low Clock Frequency

The derived clock requirement ( $\approx 1.5$  kHz) is artificially low because the design is fully bandwidth-bound.

Real hardware can operate much faster, but compute units will remain idle unless memory bandwidth increases.

### 5.3 Extending to FP32

The same scheduling and control structure can be applied to FP32, but:

- FP units have deeper pipelines (3–6 stages).
- Accumulators require exponent alignment + normalization.
- Memory footprint grows 4 $\times$ , making the 16-byte SRAM insufficient.  
Conceptually the flow maps over, but the hardware cost grows significantly.

### 5.4 Resource Summary

- Compute: 4 MACs ( $8 \times 8 \rightarrow 16$ -bit mult, 20-bit ACC, MAC/ADD modes).
- Memory: 16-byte dual-port SRAM (8 B/cycle total), external 1 KB/s interface.
- Performance: 24 cycles/patch  $\rightarrow$  384 cycles/image  $\rightarrow \approx 4$  images/s (bandwidth-limited).